

# Demo Script

---

## Demo Script (5 min)

---

One line per beat. Say it, do the action, move on. Don't elaborate.

**Covers the four required elements:** 1. build decisions · 2. tradeoffs · 3. measured results · 4. next iteration

---

### Before recording

```
cd ~/Documents/Capstone_FDE_Project && source .venv/bin/activate  
rm -rf data/proposals data/decisions.json audit_log.jsonl  
python data/generate_packets.py  
.venv/bin/streamlit run app/review_app.py
```

Slides full screen on slide 1 · browser at localhost:8501 · `data/faxes/` open in Finder · notifications off.

---

### 0:00 • SLIDE 1

“Hi, I’m Vidhanshee. I built an intake agent for prior authorization. It catches everything missing in one pass, so providers aren’t chased three times for three different things.”

### 0:12 • SLIDE 2 — the problem

“Requests arrive incomplete, the insurer writes back, days pass, it comes back still incomplete. Catching a missing test on day one costs nothing. Finding it on day twelve, after a denial and an appeal, costs a lot.”

## 0:30 • SLIDE 3 — 1. build decisions

“My main decision: rules do all the checking, AI only reads language. Everything on the left is plain code, including every routing decision, because a regulator has to be able to read the rule behind it.”

## 0:50 • SLIDE 4 — architecture

“The whole system. The three orange boxes are the only places a model is used: reading a fax, judging the notes, drafting the letter. Each one has a rules-based fallback, so it still runs end to end without an API key, just on rules instead of AI.”

## 1:05 • SLIDE 5 → switch to browser

“Let me show you.”

## 1:10 • One command

```
python -m src.pipeline PA-2026-0018
```

“This drug needs a negative TB test. The notes never mention it. Keyword search misses that, the model catches it, and a rubric in code makes sure the letter asks for exactly that and nothing more.”

## 1:45 • The human gate

**Click a case with findings:** > “Nothing leaves without this screen. Each finding is labelled rule or AI.”

**Edit one line of the letter:** > “Edits are captured. That’s labelled signal for where the AI fell short.”

**Click Approve:** > “Logged with name and timestamp, append-only.”

**Upload panel** → **drag in** `FAX-0001.txt` : > “A raw fax: extracted, checked, routed, queued.”

**Refresh queue** → **Dashboard tab:** > “Every model call is metered against a budget. If it ever looped, it falls back to rules instead of spending money.”

---

## 2:45 • 3. Measured results

**Evals tab** → “Run evaluation now”

“Twenty-nine cases, fifteen failure types, twenty-seven tests. The answer key is only readable by the scorer, never the pipeline.”

**Slow down here:**

“My favourite number came from a mistake. An early AI run scored a perfect hundred, but rate limiting meant my own fallback was quietly answering half the calls. The eval wasn’t testing the AI at all.

So I added call integrity: how many calls the model actually answered. An eval that can’t spot its own dilution isn’t an eval.”

---

## 3:40 • 2. Tradeoffs and 4. next

“Left out on purpose: image scanning, a full policy catalogue, and auto-approval for trusted providers.

Next is the loop this was built for. Every reviewer edit is labelled signal, fed back with the eval suite as the safety net.”

“That’s the intake agent. One-minute setup in the repo, and you can run it with a model or on rules alone. Thanks for watching.”

## If you’re running long

Cut in this order: the upload demo → the dashboard line → the tradeoffs middle sentence.  
Never cut: the day-one insight · rules vs AI · the human gate · the integrity story.

## If it breaks

| Problem                      | Do this                                                                          |
|------------------------------|----------------------------------------------------------------------------------|
| Command fails or rate-limits | Add <code>PA_MODE=offline</code> in front. Say “that’s the fallback I mentioned” |
| App won’t load               | <pre>.venv/bin/streamlit run app/<br/>review_app.py --server.port 8502</pre>     |
| Odd error                    | You’re on Anaconda’s Python. Use <code>.venv/bin/streamlit</code>                |

## The 6 routes (if asked)

First match wins. Only the last two are success paths.

| # | Route                                                     | When                                                    |
|---|-----------------------------------------------------------|---------------------------------------------------------|
| 1 | <code>no_auth_required</code>                             | Doesn't need prior auth                                 |
| 2 | <code>duplicate_review</code>                             | Same member + procedure in 14 days                      |
| 3 | <code>eligibility_review</code>                           | Member not in eligibility. Our team, not the provider's |
| 4 | <code>provider_outreach</code>                            | Provider must supply something. Letter drafted          |
| 5 | <code>expedited_clinical_review::&lt;specialty&gt;</code> | Complete + justified urgency                            |
| 6 | <code>clinical_review::&lt;specialty&gt;</code>           | Complete. Standard review                               |

## If they ask

**Why not LangGraph?** Straight line with one pause. A framework replaces twenty lines with a dependency.

**Why not let AI route?** Couldn't certify it. A regulator needs to read the rule.

**Letters not hallucinated?** Code rubric on every draft: findings covered, nothing invented, case ID present. One retry, then a template.

**Where's the OCR?** Deliberately none. My faxes are text, so I built the harder half. A scanner slots in front.

**Isn't 100% suspicious?** It should be. It's a consistency check against labels made by construction. The honest number is the integrity metric that caught my own eval lying.